From Static Architectures to Evolutionary Neural Systems: A Bio-Inspired Approach to Artificial General Intelligence

Devanik Debnath
National Institute of Technology Agartala

First published as a GitHub preprint on October 31, 2025

Abstract

Modern artificial intelligence systems are dominated by fixed architectural families: Transformers, recurrent networks, graph neural networks, mixture-of-experts models, memory-augmented networks, and increasingly elaborate hybrids. These architectures have produced remarkable progress, yet each embodies a strong prior over computation. The result is an architectural selection problem: researchers must choose, hand-design, scale, and tune a topology before learning can begin, even though the appropriate structure may depend on the task distribution, the available compute, the desired adaptation regime, and the kinds of generalization required. This paper develops a comprehensive framework for *morphogenetic neural architectures*: neural systems in which architecture is encoded as an evolvable genotype, expressed through a developmental genotype–phenotype map, and refined by intra-lifetime learning. The framework integrates three nested timescales: phylogenetic evolution over populations of genotypes, ontogenetic development that grows executable phenotypes from compact programs, and experiential learning through gradient descent, local plasticity, and neuromodulated update rules. We formalize genotype components, mutation and recombination operators, developmental programs, morphogenetic gradients, plasticity genes, multi-objective selection, speciation, novelty search, and quality-diversity optimization. We then analyze expressivity, compression, evolvability, sample complexity, convergence limitations, and the Baldwin interaction between learning and evolution. The central thesis is not that any single hand-designed architecture is the path to artificial general intelligence, but that architecture should itself become a learned, heritable, recombinable, and developmentally expressed object of optimization. The paper concludes with an implementation blueprint, an empirical validation protocol, limitations, and a research roadmap for transforming architecture design from manual selection into open-ended discovery.

Keywords: neuroevolution; genotype–phenotype mapping; neural architecture search; developmental encoding; morphogenetic computation; meta-learning; continual learning; artificial general intelligence.

Citation note: This work was first published by Devanik Debnath as a GitHub preprint on October 31, 2025, under the title *From Static Architectures to Evolutionary Neural Systems: A Bio-Inspired Approach to Artificial General Intelligence*.

Contents

1 Introduction

Deep learning has largely treated the neural architecture as a human-designed container for learning. Once the container is chosen, parameters are optimized by data-driven training. This separation has been extraordinarily productive: convolutional networks codified translation equivariance for visual perception, recurrent networks codified temporal recurrence, Transformers codified content-addressed sequence processing, graph neural networks codified relational message passing, and mixture-of-experts systems codified conditional computation. Yet the success of these designs also exposes a limitation. Each architectural family answers the question "what computation should be easy?" before the system has encountered the full range of environments in which it must act.

The problem becomes sharper for systems intended to display broad, adaptive intelligence. A generally capable agent must learn quickly from few examples, retain old knowledge while acquiring new skills, transfer abstractions across domains, combine primitives compositionally, reason over causal structure, integrate multiple modalities, and operate under resource constraints. A single static topology may be too rigid to support all of these regimes. Scaling a fixed architecture can increase capacity, but it does not by itself solve architectural mismatch, catastrophic forgetting, or the need for open-ended structural innovation.

Biological intelligence suggests an alternative. Brains are not specified as complete connection matrices. They arise from compact genetic encodings, developmental programs, morphogen gradients, activity-dependent refinement, and lifetime plasticity. Evolution does not directly write every synapse;

it discovers developmental rules that reliably produce adaptive organisms. This distinction is crucial. A genome is not a final brain. It is a compressed, heritable program for growing a brain that can subsequently learn.

This paper translates that biological lesson into an artificial intelligence framework. We propose that neural architecture should be represented as a genotype, developed into a phenotype, trained during its lifetime, evaluated across task distributions, and selected or recombined across generations. Under this view, Transformers, recurrent networks, graph networks, attention mechanisms, external memories, differentiable plasticity rules, and expert-routing systems are not mutually exclusive architectural choices. They are possible phenotypes, or substructures within phenotypes, reachable from a richer space of genetic programs.

The framework has four objectives. First, it dissolves the forced choice among fixed architectures by embedding them in a common morphospace. Second, it provides a compact developmental representation that can express large, regular, modular systems without directly encoding every parameter. Third, it integrates learning and evolution rather than treating them as competing paradigms. Fourth, it supports open-ended discovery by preserving diversity, rewarding novelty, and selecting for multiple objectives beyond benchmark accuracy.

1.1 Contributions

This paper makes the following contributions:

1. It formalizes a genotype–phenotype representation for neural architectures, including module genes, connection genes, plasticity genes, developmental genes, regulatory genes, and objective genes.
2. It defines mutation, recombination, speciation, novelty, and quality-diversity mechanisms suitable for evolving neural systems rather than merely tuning hyperparameters.
3. It develops a developmental mapping in which compact genotypes produce executable neural phenotypes through staged growth, morphogenetic gradients, recursive expansion, activity-dependent pruning, and canalization.
4. It integrates three adaptation timescales: evolution across generations, development within an individual architecture, and learning during task experience.
5. It analyzes expressivity, compression, modularity, evolvability, and limitations, emphasizing what can be claimed theoretically and what requires empirical validation.
6. It provides a publishable experimental blueprint for evaluating morphogenetic neural architectures against fixed architectures, neural architecture search, meta-learning, and continual-learning baselines.

2 Motivation: The Architectural Selection Problem

The dominant workflow in deep learning is sequential: choose an architecture, instantiate parameters, train on data, evaluate, and then revise the design if performance is inadequate. This workflow

leaves architecture outside the main learning loop. Hyperparameter search and neural architecture search partially automate the process, but they often search within a constrained template supplied by the researcher. The search space may include layer widths, operation choices, skip connections, or routing decisions, while the deeper grammar of architecture remains fixed.

The architectural selection problem has three components. The first is *combinatorial*: the number of plausible architectural motifs grows quickly as researchers introduce attention, recurrence, memory, sparsity, routing, modularity, differentiable stacks, graph propagation, and world-model components. The second is *distributional*: architectures are usually optimized for benchmark distributions that may not match deployment conditions. The third is *temporal*: a static architecture may be appropriate early in training but inadequate after the agent encounters new tasks, modalities, or constraints.

Morphogenetic neural architectures respond by moving architectural choice into the adaptive process. Instead of asking which topology should be selected by the human designer, we ask which genetic language, developmental mapping, learning rules, and selection pressures allow useful topologies to emerge.

2.1 Failures of Static Topology

Static architectures can fail in several ways relevant to general intelligence. They may forget previous tasks when trained sequentially, because parameter updates overwrite representations shared across tasks. They may require many examples because they lack inherited priors for rapid adaptation. They may generalize poorly outside the training distribution because learned functions are entangled with surface statistics. They may scale inefficiently because every input activates most of the same computation. Finally, they may be difficult to adapt when a new domain requires qualitatively different computational structure.

These failures are not merely optimization problems. They are symptoms of a deeper rigidity: structure is fixed while experience changes. A system that can alter its routing, memory, plasticity, modular boundaries, or developmental expansion has more ways to respond than a system whose only adaptation mechanism is weight adjustment.

2.2 Biological Analogy and Its Limits

The biological analogy is productive but must be used carefully. Artificial genotypes need not mimic DNA literally, and artificial development need not reproduce embryology. The essential abstraction is a separation between a compact heritable description and the grown computational organism. The genotype biases and constrains development; the phenotype acts and learns; selection favors genotypes whose developmental products adapt well. This abstraction captures the computational advantage of indirect encoding without requiring biological realism in every detail.

3 Related Work

Morphogenetic neural architectures synthesize several research traditions.

Neuroevolution evolves neural weights, topologies, or learning rules. NEAT demonstrated that augmenting topology during evolution can discover compact networks while protecting innovation through speciation [1]. HyperNEAT and compositional pattern-producing networks extended this idea with indirect encodings capable of generating regular connectivity patterns [2].

Neural architecture search automates aspects of architecture design using reinforcement learning, evolutionary search, differentiable relaxation, or weight sharing. Regularized evolution demonstrated strong image-classifier search performance [3]. DARTS made architecture search differentiable by relaxing discrete operation choices [4]. These methods typically search within a predefined cell or macro-architecture, whereas morphogenetic systems seek a broader developmental grammar.

Meta-learning optimizes models for rapid adaptation. MAML learns initial parameters that adapt quickly under gradient descent [5]; Reptile provides a simpler first-order alternative [6]. In the present framework, meta-learning can occur at the level of initial weights, architecture, plasticity rules, and developmental schedules.

Continual learning addresses catastrophic forgetting through regularization, replay, modular expansion, dynamic routing, or parameter isolation. Morphogenetic architectures contribute by making modularity, gating, and memory allocation evolvable rather than fixed.

Differentiable plasticity and neuromodulation allow networks to learn update rules beyond standard gradient descent. Differentiable plasticity treats plasticity coefficients as trainable parameters [7]; neuromodulated plasticity introduces context-dependent learning rates and local update rules.

Open-endedness and quality diversity emphasize novelty, diversity, and stepping-stone discovery rather than a single scalar objective. Novelty search showed that abandoning fixed objectives can sometimes discover solutions unreachable by direct optimization [8]. Quality-diversity algorithms such as MAP-Elites retain diverse high-performing solutions across behavioral niches [9].

The proposed framework is best understood as a unification: architecture search becomes evolutionary; evolutionary search becomes developmental; development becomes plastic; and plasticity is evaluated across broad task distributions.

4 Formal Framework

Let $\mathcal{G}$ denote the genotype space, $\mathcal{P}$ the phenotype space of executable neural systems, $\mathcal{T}$ a task distribution, and $\Phi : \mathcal{G} \to \mathcal{P}$ a developmental map. A genotype $g \in \mathcal{G}$ develops into a phenotype $p = \Phi(g)$. During its lifetime, p interacts with tasks sampled from $\mathcal{T}$ and updates internal state or parameters by a learning operator $\mathcal{L}$. A fitness functional F evaluates the post-learning behavior of the phenotype:

$$F(g) = \mathbb{E}_{\tau \sim \mathcal{T}} \left[U \left(\mathcal{L}(\Phi(g), \tau), \tau \right) \right] - \lambda C(\Phi(g)), \tag{1}$$

where U measures utility and C measures computational, memory, energy, or latency cost.

Evolution operates on a population $\mathcal{P}_G^{(k)} = \{g_1^{(k)}, \ldots, g_N^{(k)}\}$ at generation k. Selection, mutation, recombination, and diversity mechanisms transform this population into the next generation. The

system is therefore a nested optimization process:

$$\underbrace{g}_{\text{evolution}} \xrightarrow{\Phi} \underbrace{p}_{\text{development}} \xrightarrow{\mathcal{L},\tau} \underbrace{p'}_{\text{learning}} \xrightarrow{F} \underbrace{\text{selection}}_{\text{population update}} . \tag{2}$$

Definition 1 (Genotype)**.** *A genotype is a finite, heritable program specifying architectural primitives, connection rules, developmental schedules, plasticity mechanisms, regulatory controls, and evaluation-relevant constraints.*

Definition 2 (Phenotype)**.** *A phenotype is an executable neural system produced by development from a genotype. It includes network topology, parameters or parameter initializers, routing functions, memory systems, plasticity coefficients, and update rules.*

Definition 3 (Developmental map)**.** *A developmental map* Φ *is an algorithm that transforms a genotype into a phenotype through staged expansion, module instantiation, connection formation, pruning, specialization, and optional activity-dependent refinement.*

5 Genotype Design

A useful artificial genotype must be expressive enough to encode known architectures, compact enough for efficient search, modular enough for recombination, and stable enough that small mutations often produce viable phenotypes. We define six classes of genes.

5.1 Module Genes

Module genes specify computational primitives. A module gene may encode an attention block, convolutional block, recurrent cell, graph-message-passing layer, expert network, memory interface, normalization operator, gating unit, differentiable stack, or symbolic interface. Formally, a module gene is

$$m = (\text{type}, \theta, \rho, \pi), \tag{3}$$

where θ contains hyperparameters, ρ contains resource constraints, and π contains plasticity or initialization metadata.

Module genes should not be limited to currently fashionable building blocks. The goal is to define a computational alphabet from which evolution can discover both familiar and unfamiliar structures. A Transformer becomes a repeated arrangement of attention and feed-forward module genes; a mixture-of-experts model becomes a set of expert module genes plus routing genes; a recurrent network becomes recurrence-bearing module and connection genes.

5.2 Connection Genes

Connection genes specify information flow between modules. A connection gene is

$$c = (i, j, \alpha, \gamma, \delta), \tag{4}$$

where i and j identify source and target modules, α specifies connection strength or initialization, γ specifies gating or routing conditions, and δ specifies delay, recurrence, or temporal semantics.

Connection genes can encode dense links, sparse links, residual paths, skip connections, recurrent loops, attention masks, routing edges, cross-modal bridges, and memory read/write channels. The connection graph is not assumed to be feed-forward unless the developmental program imposes that constraint.

5.3 Plasticity Genes

Plasticity genes define how parts of the phenotype change during experience. They may encode gradient-based learning rates, local Hebbian coefficients, anti-Hebbian terms, eligibility traces, neuromodulatory gates, consolidation schedules, and rules for synaptic growth or pruning. A general local plasticity rule can be written as

$$\Delta w_{ij} = \eta_{ij} M(t) \left(a_i a_j - \beta w_{ij} + \xi e_{ij} \right), \tag{5}$$

where a_i and a_j are activities, $M(t)$ is a modulatory signal, e_{ij} is an eligibility trace, and η_{ij}, β, ξ are evolved or learned coefficients.

Plasticity genes are central because they allow evolution to discover not only what the model is at initialization, but how it should learn. A system can evolve stable modules, highly plastic modules, fast-adapting memory, or protected knowledge stores depending on the task distribution.

5.4 Developmental Genes

Developmental genes specify how the genotype expands into a phenotype. They include repetition counts, growth stages, symmetry rules, scaling laws, morphogen gradients, recursion templates, pruning thresholds, and activity-dependent refinement procedures. Indirect developmental encoding allows a compact genotype to express a large structured phenotype.

5.5 Regulatory Genes

Regulatory genes control when other genes are expressed. They can encode context-dependent module activation, developmental stage transitions, resource-sensitive scaling, or task-conditioned routing. Regulatory genes make the genotype more than a list of components; they create a grammar of conditional expression.

5.6 Objective Genes

Objective genes specify evaluation priorities such as accuracy, sample efficiency, forgetting resistance, calibration, robustness, energy use, latency, interpretability, novelty, and transfer. In biological evolution, the environment supplies selection. In artificial evolution, the designer must specify or learn the selection pressures. Objective genes make these pressures explicit and potentially evolvable.

6 Developmental Morphogenesis

The developmental map Φ is the bridge between genotype and phenotype. It should satisfy four desiderata: compactness, regularity, viability, and diversity.

6.1 Staged Development

Development proceeds through stages. A minimal schedule is:

1. **Specification**: decode genes and establish global constraints.
2. **Proliferation**: instantiate modules according to repetition, recursion, and scaling rules.
3. **Patterning**: assign positional identities using morphogenetic coordinates or graph roles.
4. **Connection**: create candidate edges using connection genes and positional rules.
5. **Pruning**: remove weak, redundant, invalid, or resource-violating components.
6. **Specialization**: initialize module-specific plasticity, routing, and memory systems.
7. **Canalization**: enforce stability constraints so similar genotypes produce functionally coherent phenotypes.

6.2 Morphogenetic Gradients

Morphogenetic gradients provide positional information. In an artificial architecture, each module can be assigned coordinates $x_i \in \mathbb{R}^d$. Connection probability can depend on distance, module type, developmental stage, and learned compatibility:

$$\Pr(c_{ij} = 1) = \sigma\left(f_\psi(x_i, x_j, m_i, m_j, s)\right), \tag{6}$$

where s is the developmental stage and f_ψ is a genetically specified or learned compatibility function. This mechanism can generate locality, hierarchy, repeated motifs, bilateral symmetry, hub structure, sparse modularity, and long-range integration.

6.3 Recursive Programs and Compression

Large architectures often contain repeated structure. A genotype can encode "repeat this block L times," "grow a hierarchy until the resource budget is reached," or "expand experts around task niches." This yields developmental compression.

Definition 4 (Genetic complexity)**.** *The genetic complexity $\kappa(p)$ of a phenotype p is the length of the shortest genotype that develops into a functionally equivalent phenotype:*

$$\kappa(p) = \min\{|g| : \Phi(g) \approx_\epsilon p\}. \tag{7}$$

Proposition 1 (Developmental compression)**.** *For a regular network with L repeated layers and representation dimension D, an indirect genotype can encode the repetition and module type in $O(\log L + \log D)$ structural description length, while direct specification of dense layer matrices requires $O(LD^2)$ parameter entries.*

Proof. The genotype stores a block template, the representation dimension, and a repetition count. The repetition count requires logarithmic description length in L, and the dimension requires logarithmic description length in D under standard integer encodings. Direct specification stores each layer matrix, giving L matrices of size proportional to D^2. The comparison concerns structural description length, not the trained numerical parameter values. □

6.4 Activity-Dependent Development

Development can include a short self-organization phase in which synthetic inputs or early task exposure refine connectivity. Candidate connections may be strengthened when useful, pruned when inactive, or gated when interfering. This resembles biological activity-dependent refinement while remaining computationally implementable.

6.5 Canalization and Robustness

An evolvable system must tolerate mutation. Canalization means that many small genetic perturbations produce viable phenotypes. Artificial canalization can be encouraged by type constraints, resource budgets, normalization, repair operators, developmental defaults, modular boundaries, and penalties for brittle phenotypes. Without canalization, evolution degenerates into random architecture destruction.

7 Evolutionary Dynamics

Evolutionary search updates a population of genotypes. The goal is not merely to maximize a scalar benchmark score, but to discover a diverse archive of architectures that adapt well across tasks and constraints.

7.1 Mutation Operators

Mutation operators include parametric, topological, plastic, and developmental changes. Parametric mutations perturb widths, depths, kernel sizes, attention heads, learning rates, regularization coefficients, routing temperatures, or memory capacities. Topological mutations add or remove modules, add or remove connections, duplicate subgraphs, split modules, merge modules, alter recurrence, or introduce routing gates. Plasticity mutations change learning rules, local update coefficients, consolidation rates, or neuromodulatory pathways. Developmental mutations alter growth schedules, repetition counts, morphogen gradients, pruning thresholds, or recursion templates.

Viability-preserving mutation is essential. Mutations should be type-checked, budget-aware, and repairable. For example, if a connection mutation creates incompatible tensor dimensions, the developmental system may insert a projection module rather than discarding the genotype.

7.2 Recombination Operators

Recombination exchanges genetic material between parents. Effective recombination should preserve functional modules. Module-level crossover identifies subgraphs with high internal connectivity and low external dependency, then transfers them as units. Hyperparameter blending interpolates continuous parameters. Innovation tracking aligns homologous genes so crossover combines corresponding structures rather than randomly splicing incompatible components.

7.3 Selection

Single-objective selection is inadequate for general intelligence. A scalar score can favor shortcut solutions, over-specialized architectures, excessive compute, or brittle benchmark exploitation. We therefore define a vector fitness:

$$\mathbf{F}(g) = (F_{\text{task}}, F_{\text{transfer}}, F_{\text{continual}}, F_{\text{few-shot}}, F_{\text{robust}}, -C_{\text{compute}}, F_{\text{novelty}}) \,. \tag{8}$$

Selection can use Pareto dominance, lexicographic constraints, scalarization with adaptive weights, or quality-diversity archives.

7.4 Speciation and Novelty

Speciation protects innovation by comparing genotypes or phenotypes using a compatibility distance. New structures often perform poorly before refinement; without protection, they disappear before their potential is realized. Novelty search rewards behavioral difference from an archive, allowing stepping stones that do not immediately improve the main objective.

7.5 Quality Diversity

Quality-diversity algorithms maintain elites across niches. A niche may be defined by architectural descriptors such as parameter count, recurrence depth, modularity, memory capacity, sparsity, routing entropy, or plasticity level. The result is not one winner but a map of high-performing architectures across design regimes. This is especially useful when deployment environments impose different latency, memory, or adaptation constraints.

8 Learning Across Three Timescales

Morphogenetic neural architectures integrate phylogenetic, ontogenetic, and experiential adaptation.

8.1 Phylogenetic Evolution

Evolution searches over genotypes. It is slow but global: it can alter architecture, learning rules, developmental programs, and priors. Evolution answers the question: what kind of learner should be born?

8.2 Ontogenetic Development

Development transforms a genotype into a phenotype. It is medium-speed and structured: it grows modules, establishes routing, initializes plasticity, and imposes constraints. Development answers the question: how should the inherited program become an individual model?

8.3 Experiential Learning

Learning updates the phenotype during tasks. It is fast and local relative to evolution. It may include backpropagation, local plasticity, memory writes, adapter updates, routing adaptation, and consolidation. Learning answers the question: how should this individual respond to experience?

8.4 Baldwin Effect

The Baldwin effect describes how learning can guide evolution. If individuals with certain genotypes learn useful behavior more easily, selection favors those genotypes even before the behavior is genetically fixed. Over generations, evolution may assimilate parts of the learned behavior into architecture or initialization. In artificial systems, this suggests selecting for rapid learnability rather than only final performance.

9 Theoretical Properties

9.1 Architectural Subsumption

If the genotype language includes module and connection primitives sufficient to express attention, recurrence, convolution, graph propagation, memory, and routing, then many existing architectures appear as special cases. A Transformer is a genotype with repeated self-attention and feed-forward modules. A mixture-of-experts system is a genotype with expert modules and learned routers. A recurrent network is a genotype with temporal connection genes. A graph neural network is a genotype whose connections follow graph message-passing rules.

Theorem 1 (Subsumption)**.** *Let $\mathcal{A}$ be a finite set of neural architecture families. If the genotype language contains primitives and developmental rules that instantiate every operation and connectivity pattern used by each family in $\mathcal{A}$, then every architecture in $\mathcal{A}$ is representable as a phenotype of some genotype $g \in \mathcal{G}$.*

Proof. For each architecture family, map its operations to module genes and its edges to connection genes. Encode repetition, depth, and width through developmental genes. Because the language contains the required primitives and connectivity rules by assumption, the developmental map can instantiate the architecture exactly or to arbitrary implementation precision. □

9.2 Universal Expressivity

Universal expressivity follows under standard conditions: if the phenotype language can instantiate known universal approximators, then the morphogenetic system can instantiate universal approximators. This does not mean evolution will efficiently find them, nor that universal approximation implies intelligence. It only establishes that the representation is not inherently less expressive than fixed architectures.

9.3 Evolvability

Evolvability is the capacity to generate heritable, selectable variation. It depends on modularity, redundancy, smooth genotype–phenotype neighborhoods, and recombinable substructures. Developmental encodings can increase evolvability because they allow mutations to affect motifs, scaling rules, or plasticity regimes rather than isolated parameters.

9.4 Sample Complexity and Surrogates

The main cost of evolutionary architecture search is evaluation. Training every phenotype to convergence is usually infeasible. Practical systems require surrogate predictors, low-fidelity evaluations, weight inheritance, early stopping, curriculum tasks, shared training environments, and distributed evaluation. A surrogate $\hat{F}_\omega(g)$ can prioritize candidates, but it must be calibrated and periodically corrected with true evaluations to avoid model bias.

9.5 Convergence Limits

No general theorem guarantees efficient convergence to globally optimal architectures across arbitrary task distributions. The search space is vast, evaluation is noisy, and objectives may conflict. Therefore, the framework should be understood as a principled search and discovery process, not as a promise of guaranteed optimality. Strong claims require empirical evidence on clearly defined distributions.

10 Implementation Blueprint

An implementation should separate genotype representation, development, training, evaluation, and population management.

10.1 Genotype Schema

Each genotype can be represented as a typed graph:

$$g = (M, C, P, D, R, O), \tag{9}$$

where M is the set of module genes, C connection genes, P plasticity genes, D developmental genes, R regulatory genes, and O objective genes. Type constraints ensure that the graph can develop into an executable model.

10.2 Development Engine

The development engine performs decoding, graph expansion, compatibility checking, shape inference, resource accounting, module initialization, and optional repair. It should emit both the executable phenotype and metadata describing architecture descriptors, lineage, mutation history, and developmental decisions.

10.3 Training Engine

The training engine supports standard gradient descent, local plasticity, adapter learning, replay buffers, task-conditioned routing, and consolidation. It records learning curves rather than only final accuracy, because rapid adaptation and resistance to forgetting are core fitness dimensions.

10.4 Evolution Engine

The evolution engine maintains a population and archive. Each generation performs candidate selection, mutation, recombination, development, low-fidelity screening, training, evaluation, archive update, and survivor selection. Parallel evaluation is essential.

10.5 Reproducibility

A publishable system must log random seeds, genotype hashes, developmental traces, task samples, training budgets, hardware, evaluation protocols, and all selection decisions. Evolutionary systems are especially vulnerable to irreproducibility because small stochastic differences can alter lineages.

11 Empirical Validation Protocol

This paper proposes the following validation program. Results should be reported only after controlled experiments are run; until then, the framework should be treated as a theoretical and methodological contribution.

11.1 Benchmarks

A comprehensive suite should include:

- **Compositional reasoning**: tasks requiring systematic recombination of primitives.
- **Continual learning**: sequential tasks measuring retention, transfer, and forgetting.
- **Few-shot learning**: adaptation from limited examples.
- **Abstract reasoning**: symbolic or relational tasks such as program induction and matrix reasoning.
- **Algorithmic tasks**: copying, sorting, graph traversal, arithmetic, and variable binding.
- **Multimodal tasks**: integration of text, vision, action, and structured state.
- **Out-of-distribution robustness**: distribution shifts, corruptions, and changed task compositions.

11.2 Baselines

Baselines should include fixed Transformers, recurrent models, graph networks, mixture-of-experts systems, neural architecture search methods, meta-learning methods, modular continual-learning methods, and ablated morphogenetic variants. Ablations should remove development, plasticity, speciation, novelty search, recombination, and multi-objective selection independently.

11.3 Metrics

Accuracy alone is insufficient. Metrics should include final performance, sample efficiency, adaptation speed, forgetting, forward transfer, backward transfer, compute cost, memory cost, calibration, robustness, architectural novelty, modularity, sparsity, routing entropy, and lineage diversity.

11.4 Statistical Reporting

Evolutionary experiments should report multiple independent runs, confidence intervals, effect sizes, lineage visualizations, archive coverage, and compute budgets. Because search can be compute-intensive, comparisons should be budget-matched as well as performance-matched.

12 Applications to AGI Challenges

12.1 Continual Learning

Catastrophic forgetting arises when new learning overwrites old representations. Morphogenetic systems can evolve protected modules, replay pathways, expandable subnetworks, consolidation

rules, and task-conditioned gates. Rather than imposing one continual-learning algorithm, evolution can discover the balance between stability and plasticity appropriate to the task distribution.

12.2 Compositional Reasoning

Compositional reasoning benefits from modular representations and reusable operations. Developmental encodings naturally support repeated motifs and hierarchical organization. Selection across compositional tasks can favor architectures that separate primitives, relations, and binding mechanisms.

12.3 Few-Shot Adaptation

Few-shot learning requires strong priors and fast update mechanisms. Evolution can encode initial architectures and plasticity rules that make useful hypotheses easy to reach from few examples. The objective should reward learning curves, not only final performance.

12.4 Causal and Abstract Reasoning

Causal reasoning may require interventions, variable abstraction, counterfactual simulation, and modular world models. A morphogenetic search space can include memory, graph, symbolic, and simulation modules, allowing architectures to specialize for causal structure when selected by appropriate tasks.

12.5 Open-Ended Learning

Open-ended learning requires preserving stepping stones, not only optimizing a fixed objective. Novelty search, quality diversity, and environment co-evolution can help maintain exploration. The key design challenge is to create selection pressures that reward increasing competence without collapsing diversity.

13 Philosophical and Methodological Implications

The framework shifts AI research from design to discovery. The human role becomes defining primitives, developmental laws, safety constraints, task ecologies, and selection pressures. The machine role becomes discovering architectures within that space.

This does not eliminate human responsibility. Selection pressures encode values and incentives. If they reward benchmark exploitation, systems may discover shortcuts. If they reward capability without constraints, systems may become difficult to interpret or control. Therefore, morphogenetic architecture search must be coupled to transparency, auditing, safety evaluation, and constraint design.

The framework also reframes the debate over innate structure versus learning. In a morphogenetic system, innate structure is not opposed to learning; it is what evolution discovers to make learning possible. A useful genotype does not solve every task directly. It creates a learner with appropriate inductive biases.

14 Limitations and Risks

The approach has substantial limitations. First, evaluation is expensive because each genotype may require development and training. Second, search spaces can be deceptive; evolution may exploit proxies rather than solve intended tasks. Third, evolved architectures may be difficult to interpret. Fourth, reproducibility is challenging because lineages depend on stochastic events. Fifth, claims about AGI remain speculative until supported by rigorous evidence. Sixth, multi-objective optimization introduces value choices that cannot be hidden behind technical language.

Practical deployments should begin with small, controlled domains; use strict budget matching; publish failed runs; maintain interpretable lineage records; and avoid overstating empirical performance before experiments are complete.

15 Research Roadmap

15.1 Near Term

Near-term work should implement typed genotype graphs, robust development engines, small-scale task suites, surrogate fitness predictors, and ablation studies. The goal is to show that developmental encoding and plasticity improve search efficiency or adaptation under controlled budgets.

15.2 Medium Term

Medium-term work should scale to larger models, distributed evolution, quality-diversity archives, continual-learning benchmarks, and multimodal tasks. The research emphasis should shift from isolated best models to families of architectures and the evolutionary pressures that produce them.

15.3 Long Term

Long-term work should investigate open-ended environments, co-evolving curricula, safety-constrained selection, interpretable developmental biology of artificial architectures, and integration with foundation-model training. The eventual aim is not merely automated architecture search, but a science of artificial morphogenesis.

16 Conclusion

The central claim of this paper is that architecture should not remain a static human decision made before learning begins. It should be an evolvable, heritable, recombinable, and developmentally expressed object of optimization. Morphogenetic neural architectures provide a framework for doing so. They combine compact genetic encodings, developmental growth, plastic lifetime learning, population-based selection, novelty preservation, and multi-objective evaluation.

This framework subsumes many existing architectures as special cases while opening a larger morphospace of possible neural systems. It also clarifies the path from present-day architecture search to more open-ended artificial intelligence: define expressive genetic primitives, build reliable developmental maps, evaluate across rich task ecologies, preserve diversity, and select for adaptable learners rather than static benchmark specialists.

The future of AI may not be decided by choosing between Transformers, recurrent networks, graph networks, memory systems, or mixture-of-experts models. It may be decided by designing evolutionary and developmental processes capable of discovering when each motif is useful, how they should be combined, and how new motifs can emerge.

References

[1] K. O. Stanley and R. Miikkulainen. Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127, 2002.

[2] K. O. Stanley. Compositional pattern producing networks: A novel abstraction of development. *Genetic Programming and Evolvable Machines*, 8:131–162, 2007.

[3] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le. Regularized evolution for image classifier architecture search. *Proceedings of AAAI*, 2019.

[4] H. Liu, K. Simonyan, and Y. Yang. DARTS: Differentiable architecture search. *International Conference on Learning Representations*, 2019.

[5] C. Finn, P. Abbeel, and S. Levine. Model-agnostic meta-learning for fast adaptation of deep networks. *International Conference on Machine Learning*, 2017.

[6] A. Nichol, J. Achiam, and J. Schulman. On first-order meta-learning algorithms. arXiv:1803.02999, 2018.

[7] T. Miconi, J. Clune, and K. O. Stanley. Differentiable plasticity: Training plastic neural networks with backpropagation. *International Conference on Machine Learning*, 2018.

[8] J. Lehman and K. O. Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2):189–223, 2011.

[9] J.-B. Mouret and J. Clune. Illuminating search spaces by mapping elites. arXiv:1504.04909, 2015.

[10] J. Clune. AI-GAs: AI-generating algorithms, an alternate paradigm for producing general artificial intelligence. arXiv:1905.10985, 2019.

[11] F. Chollet. On the measure of intelligence. arXiv:1911.01547, 2019.

[12] D. Debnath. From Static Architectures to Evolutionary Neural Systems: A Bio-Inspired Approach to Artificial General Intelligence. *Preprint*, National Institute of Technology Agartala, 2025. First published on GitHub on October 31, 2025. Repository: https://github.com/Devanik21/GENEVO-GENetic-EVolutionary-Organoid.